

Deep Research: Avatrada 57 Topic 056

Engineering Report: Source Tagging Taxonomy

Report ID: ER-2023-STT-01 **Date:** October 26, 2023 **Author:** Autonomous Technical Researcher **Subject:** Deep-Dive on the Implementation of a Strict Signal Source Tagging System

Executive Summary

This report provides a detailed engineering analysis of the "Source Tagging Taxonomy" system component. The core objective is to enforce strict data provenance for every order by tagging it with its originating source. This tag must be immutable and persist throughout the entire system lifecycle, from initial signal generation to the final audit log entry.

The proposed system mandates a non-nullable `source` field in key data models (`Signal`, `Order`, `Execution`, `AuditLog`), populated from a controlled enumeration: `ALGO_GAMMA`, `ALGO_SENTIMENT`, `AI_GEMINI`, `MANUAL_USER`. This ensures that every trading action is traceable to its origin, which is critical for performance attribution, risk management, model debugging, and regulatory compliance.

This analysis deconstructs the architecture, provides a concrete implementation strategy using Python and PostgreSQL, and performs a critical analysis of potential failure modes, edge cases, and future-proof enhancements.

1. Technical Deconstruction

The system described is fundamentally a data provenance and traceability mechanism applied to a financial trading workflow. Its architecture can be

broken down into three key components: the Taxonomy, the Data Lifecycle, and the Enforcement Mechanism.

1.1. The Taxonomy: A Controlled Vocabulary

The "strict taxonomy" is implemented as an enumeration (enum). This is a fixed set of predefined constants that represent the possible sources of a trading signal.

- **ALGO_GAMMA** : Refers to a quantitative (QUANT) model. In finance, "Gamma" is a second-order Greek in options pricing, measuring the rate of change of Delta. This source likely represents a model trading options, volatility, or derivatives based on market microstructure and mathematical formulas.
- **ALGO_SENTIMENT** : Refers to a quantitative model, likely leveraging Natural Language Processing (NLP) or other machine learning techniques to analyze news, social media, or financial reports. It generates signals based on perceived market sentiment (e.g., bullish/bearish).
- **AI_GEMINI** : Explicitly names a specific Large Language Model (LLM) from Google. This source represents a signal generated by a general-purpose or fine-tuned generative AI, potentially analyzing complex, unstructured data or making predictive forecasts.
- **MANUAL_USER** : Represents a signal or order initiated directly by a human trader through a user interface. This is the catch-all for discretionary actions.

1.2. The Data Lifecycle: Tag Persistence

The critical requirement is that the `source` tag, once assigned, must be propagated immutably through every stage of the trading process.

1. **Signal Generation**: This is the point of origin. A signal object is created with a specific payload (e.g., "BUY 100 AAPL") and is **mandatorily tagged** with its source (e.g., `source: 'ALGO_SENTIMENT'`).
2. **Order Creation**: The Order Management System (OMS) consumes the signal and creates an `Order` object. The `source` tag from the `Signal` is copied directly to the `Order` object. It is not re-evaluated or changed.

3. **Execution:** The `Order` is sent to a broker and is filled (fully or partially). The resulting `Execution` record(s) must inherit the `source` tag from the parent `Order`. This allows for analyzing the performance of fills generated by different sources.
4. **Audit Log:** Every significant event in this lifecycle (Signal Created, Order Placed, Order Filled, Order Cancelled, etc.) is recorded in an `Audit Log`. Each log entry must contain the `source` tag associated with the event's subject, creating a permanent, traceable record.

This flow can be visualized as:

```
graph TD
    A["Signal Generator <br> e.g., ALGO_GAMMA"] --> B["Signal Object <br> source: ALGO_GAMMA"];
    B --> C["Order Management System"];
    C --> D["Order Object <br> source: ALGO_GAMMA"];
    D --> E["Execution Engine / Broker"];
    E --> F["Execution Record <br> source: ALGO_GAMMA"];
    B --> G["Audit Log"];
    D --> G;
    F --> G;
```

1.3. The Enforcement Mechanism

"Strictness" is enforced at multiple layers of the system:

- **Database Layer:** `NOT NULL` constraints on the `source` column in all relevant tables. The use of a database-native `ENUM` type provides an additional layer of data integrity, rejecting any write attempts with an invalid source value.
- **Application Layer:** Object models (e.g., classes, structs) for Signals, Orders, etc., will have a non-optional `source` field. This leverages the type system of the programming language to ensure a source is present at object instantiation, catching errors at compile-time or runtime before they reach the database.

- **Business Logic:** The code responsible for transforming a `Signal` into an `Order` must be explicitly programmed to carry the `source` tag forward.
-

2. Implementation Strategy

This section outlines a practical approach to building this system, using a common stack in quantitative finance: Python for application logic and PostgreSQL for the database.

2.1. Database Schema Modification (PostgreSQL)

First, we define the `ENUM` type and then alter the relevant tables to include the mandatory `source` field.

```
-- Step 1: Define the strict ENUM type for signal sources.
CREATE TYPE signal_source_enum AS ENUM (
    'ALGO_GAMMA',
    'ALGO_SENTIMENT',
    'AI_GEMINI',
    'MANUAL_USER'
);

-- Step 2: Alter the 'signals' table.
ALTER TABLE signals
ADD COLUMN source signal_source_enum NOT NULL;

-- Step 3: Alter the 'orders' table.
-- This assumes an order is always generated from a signal or manual entry.
ALTER TABLE orders
ADD COLUMN source signal_source_enum NOT NULL;

-- Step 4: Alter the 'executions' table.
ALTER TABLE executions
ADD COLUMN source signal_source_enum NOT NULL;

-- Step 5: Alter the 'audit_logs' table.
ALTER TABLE audit_logs
ADD COLUMN source signal_source_enum NOT NULL;
```

```
-- Optional: Add an index if frequent querying by source is expected.
CREATE INDEX idx_orders_source ON orders(source);
CREATE INDEX idx_executions_source ON executions(source);
```

2.2. Application Layer Implementation (Python)

We will use Python with `Pydantic` for data validation and `Enum` for type safety. This ensures that the application logic adheres to the strict taxonomy.

```
from enum import Enum
from pydantic import BaseModel, Field
from datetime import datetime
import uuid

# Step 1: Define the taxonomy in the application code, matching the database
# ENUM.
class SignalSource(str, Enum):
    """Enumeration for the source of a trading signal."""
    ALGO_GAMMA = "ALGO_GAMMA"
    ALGO_SENTIMENT = "ALGO_SENTIMENT"
    AI_GEMINI = "AI_GEMINI"
    MANUAL_USER = "MANUAL_USER"

# Step 2: Define strict data models for each stage of the lifecycle.
# Pydantic will raise a ValidationError if 'source' is missing.

class Signal(BaseModel):
    signal_id: uuid.UUID = Field(default_factory=uuid.uuid4)
    source: SignalSource # Non-optional field
    symbol: str
    action: str # e.g., 'BUY', 'SELL'
    quantity: float
    created_at: datetime = Field(default_factory=datetime.utcnow)

class Order(BaseModel):
    order_id: uuid.UUID = Field(default_factory=uuid.uuid4)
    signal_id: uuid.UUID
    source: SignalSource # Inherited, non-optional
```

```

    symbol: str
    action: str
    quantity: float
    status: str # e.g., 'PENDING', 'FILLED'

class Execution(BaseModel):
    execution_id: uuid.UUID = Field(default_factory=uuid.uuid4)
    order_id: uuid.UUID
    source: SignalSource # Inherited, non-optional
    fill_price: float
    fill_quantity: float
    executed_at: datetime

class AuditLogEntry(BaseModel):
    log_id: uuid.UUID = Field(default_factory=uuid.uuid4)
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    event_type: str # e.g., 'SIGNAL_CREATED', 'ORDER_PLACED'
    related_id: uuid.UUID
    source: SignalSource # The source of the entity being logged
    details: dict

# Step 3: Implement the business logic for propagation.
def create_order_from_signal(signal: Signal) -> Order:
    """
    Business logic to convert a Signal into an Order, ensuring source
    propagation.
    """
    # The source is explicitly and mandatorily copied.
    # It's impossible to create an Order from a Signal without the source tag.
    new_order = Order(
        signal_id=signal.signal_id,
        source=signal.source,
        symbol=signal.symbol,
        action=signal.action,
        quantity=signal.quantity,
        status='PENDING'
    )
    # ... logic to persist and submit the order ...
    # ... create AuditLogEntry('ORDER_PLACED', source=new_order.source) ...
    return new_order

```

```
# Example Usage:
# A signal is generated by the sentiment algorithm.
sentiment_signal = Signal(
    source=SignalSource.ALGO_SENTIMENT,
    symbol="TSLA",
    action="BUY",
    quantity=50
)

# The OMS processes the signal. The source tag is automatically propagated.
oms_order = create_order_from_signal(sentiment_signal)

print(f"Signal Source: {sentiment_signal.source.value}")
print(f"Order Source: {oms_order.source.value}")
# Output:
# Signal Source: ALGO_SENTIMENT
# Order Source: ALGO_SENTIMENT
```

3. Critical Analysis

While the proposed system is robust, a rigorous analysis reveals potential failure modes, edge cases, and areas for enhancement.

3.1. Potential Failure Modes and Edge Cases

- **Taxonomy Brittleness:** The use of a hardcoded `ENUM` is strict but inflexible. Adding a new algorithm (e.g., `ALGO_VOLARBITRAGE`) requires a coordinated change across the database schema, application code, and a full deployment cycle. This can slow down research and development.
- **Blended Signals:** What if a signal is generated by `ALGO_SENTIMENT` and then confirmed or modified by `AI_GEMINI` before an order is placed? The current single-value schema cannot represent this compound origin, leading to a loss of valuable metadata. The "last touch" source would get the credit, which may be misleading.

- **Manual Intervention:** A trader manually closes a position that was opened by `ALGO_GAMMA`. The closing order will be tagged `MANUAL_USER`. While correct, the system lacks a native way to link the lifecycle of the position and attribute the net P/L to the original `ALGO_GAMMA` strategy. This requires a separate concept of a `strategy_id` or `position_id` that also persists.
- **Upstream Data Corruption:** If a signal-generating service has a bug and fails to provide a `source` tag, the OMS will reject the signal. This is the desired behavior for data integrity, but it can halt trading. The system needs a robust error handling and alerting mechanism (e.g., a dead-letter queue for malformed signals) to handle such failures gracefully.

3.2. Optimizations and Enhancements

- **Flexible Taxonomy via Foreign Key:** Instead of a rigid `ENUM`, use a dedicated `signal_sources` table. `sql CREATE TABLE signal_sources ( id SERIAL PRIMARY KEY, name VARCHAR(100) UNIQUE NOT NULL, category VARCHAR(50) NOT NULL -- e.g., 'QUANT', 'AI', 'MANUAL' ); --` The 'source' column in other tables becomes an integer foreign key. `-- ALTER TABLE orders ADD COLUMN source_id INTEGER NOT NULL REFERENCES signal_sources(id);` **Pros:** New sources can be added dynamically via an API or admin interface without a code deployment. Allows for richer metadata (e.g., category). **Cons:** Loses compile-time safety of an `Enum`. Requires a `JOIN` to get the source name, a minor performance hit.
- **Support for Multiple Sources:** To address the "Blended Signals" problem, replace the single `source` field with a many-to-many relationship or a JSONB array.
 - **Many-to-Many:** Create a linking table like `order_sources (order_id, source_id)`. This is the most normalized and powerful solution.
 - **JSONB Array (PostgreSQL):** Use a `JSONB` column to store an array of source tags, e.g., `sources: [ 'ALGO_SENTIMENT', 'AI_GEMINI' ]`. This is more flexible and avoids extra `JOIN`s but can be harder to query and enforce integrity on.

- **Source Versioning:** The performance of `ALGO_GAMMA` v1.2 might be very different from v2.0. The taxonomy should support versioning. The flexible foreign key approach handles this well, where `ALGO_GAMMA_v1.2` and `ALGO_GAMMA_v2.0` would be distinct entries in the `signal_sources` table.
- **Hierarchical Tagging:** The prompt's high-level categories (`QUANT` , `AI` , `MANUAL`) are lost in the final enum. The `signal_sources` table approach (suggested above) solves this by including a `category` column, allowing for powerful aggregate analysis (e.g., "What is the total P/L for all `AI` sources vs. all `QUANT` sources?").